

TRABAJO PRÁCTICO · INTELIGENCIA ARTIFICIAL

TutorIA

Agente académico con LLM

Control de avance · 2026

¿Qué es TutorIA?

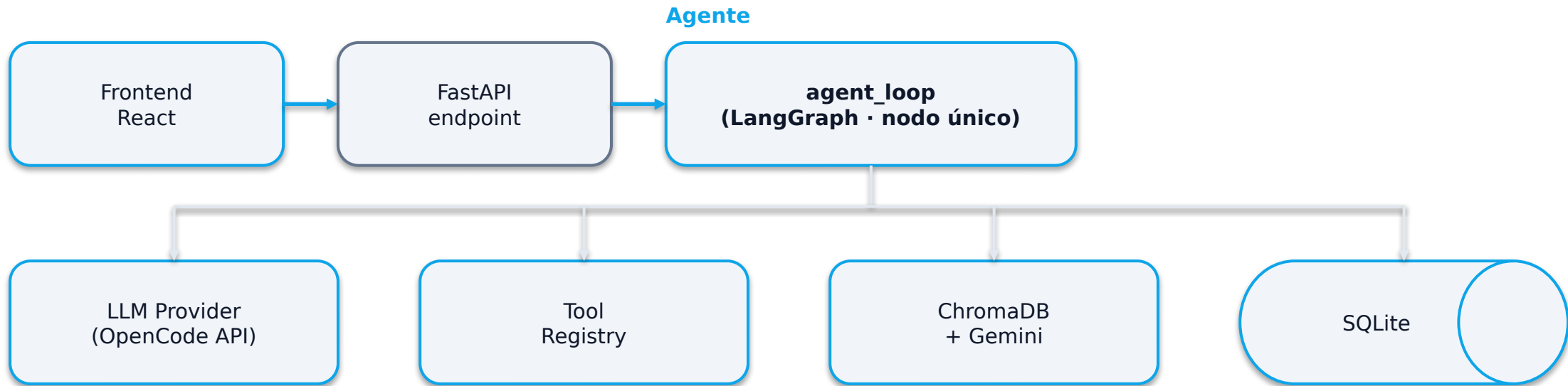
Un tutor académico para estudiantes de Ingeniería en Sistemas que conversa en lenguaje natural y se apoya en los apuntes que el estudiante sube.

Hace

- › Responde dudas conceptuales
- › Lee y cita los apuntes del estudiante
- › Evalúa y compara documentos subidos
- › Registra fechas de parciales y entregas
- › Recuerda preferencias del estudiante

Stack y conexiones externas

Tres APIs externas y un vector store local, orquestados por FastAPI.



Conexiones externas y persistencia

API OpenCode (LLM, OpenAI-compatible, 2 modelos)

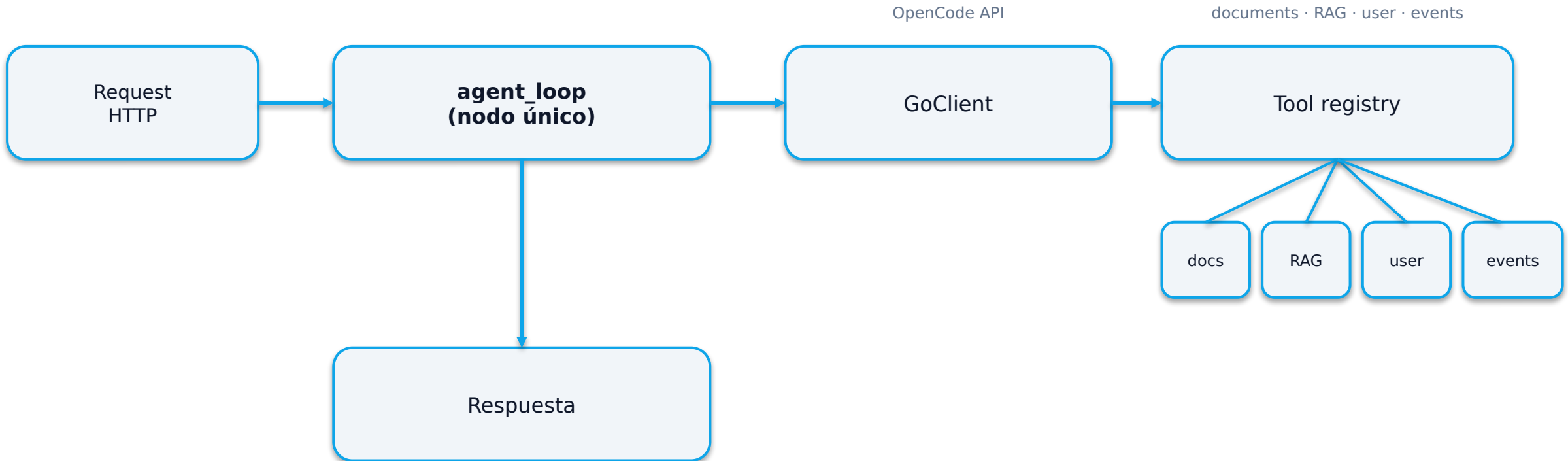
DB ChromaDB

API Google Gemini (solo embeddings)

DB SQLite (users, chats, docs, eventos)

El agente:

El StateGraph tiene un nodo. La decisión de qué hacer vive en el LLM.



Por qué un solo nodo

- › El LLM decide mejor cuándo llamar y cuándo responder si no lo encorsetamos
- › Function-calling nativo: el LLM emite tool_calls con argumentos estructurados
- › El loop interno itera hasta convergencia o cap de seguridad

Loop de razonamiento

El LLM itera y decide que tools usar. El backend ejecuta, e informa el resultado al LLM.



Cómo crece la conversación

```
iter 1: [system, user]
iter 1: + assistant(tool_calls=[read_doc, rag_search])
iter 1: + tool(read_doc) + tool(rag_search)
iter 2: [system, user, asst, tool, tool]
iter 2: → assistant(text=...) # cortamos
```

Reglas del loop

- › Cap de 5 iteraciones
- › Si no converge → devuelve lo que tenga + warning

Tools: cómo interactúa con el ambiente

8 herramientas en 4 dominios.

Categoría	Tool	Cuándo la usa el LLM
Documentos	list_documents	El usuario pregunta qué subió, o hay que decidir qué leer
	read_document	Necesita el documento entero (evaluar, comparar, analizar un TP)
RAG	rag_search	Búsqueda semántica de definiciones o secciones específicas
Usuario	get_user_memory	La pregunta depende del contexto del estudiante
	update_user_memory	El usuario cambia una preferencia persistente
Agenda	list_upcoming_events	Pregunta por fechas, próximos parciales, agenda
	set_event	El usuario informa una fecha relevante

RAG: fuentes externas (Chroma + Gemini)

Ingestión (una vez al subir el archivo)



Retrieval (cada vez que el LLM llama rag_search)



Decisiones

- › Una colección `tutoria_documents`, cosine
- › Filter server-side: aísla por materia y por usuario

Cita literal

```
[doc:enunciado.pdf, chunk:3]
El entorno del agente es la suma de
todo lo que el agente puede percibir
y sobre lo que puede actuar.
```

El LLM repite la cita tal cual en la respuesta.

Estado del agente: qué datos ve

En cada llamada, el LLM recibe un contexto acotado. El resto lo accede bajo demanda.

En el system prompt

- › Rol y reglas (no inventar, citar, etc.)
- › Carrera, año, materia, módulo
- › Catálogo de tools disponibles
- › Preferencias del estudiante
- › Últimos 12 mensajes del chat
- › Mensaje actual del usuario

Bajo demanda (vía tools)

- › Documentos del módulo / materia
- › Búsqueda semántica en apuntes (RAG)
- › Perfil completo del estudiante
- › Próximos eventos académicos
- › Documentos de otros módulos del mismo user

Fuera de su alcance

- › Datos de otros usuarios
- › Internet / APIs externas
- › Mensajes fuera de la ventana (12)
- › DB directa (toda interacción pasa por tools)

Limitaciones

- › 5 iteraciones máximo del loop (constante MAX_AGENT_ITERATIONS)
- › 200 000 chars de cap duro en read_document (~50k tokens)
- › Tool result que falla → se reporta como ok: False, el loop sigue
- › Ownership checks en cada tool (doc, evento, módulo son del usuario)
- › Whitelist de modelos y de tipos de evento

FIN